

What is Pandas in Python?

- Pandas is a Python library that is a simple yet powerful tool for Data Science. Python Pandas is one of the most widely used Python packages.
- This package comprises many data structures and tools for effective data manipulation and analysis. If we are going to work with data using Python, we need to learn Python Pandas as well.
- Python Pandas is used everywhere including commercial and academic sectors and in fields like economics, finance, analytics, statistics, etc.

Features of Python Pandas

Some of the key features of Python Pandas are as follows:

- It provides DataFrame objects with default and customized indexing which is very fast and efficient.
- There are tools available for loading data of different file formats into in-memory data objects.
- It is easy to perform data alignment and integrated handling of missing data in Python Pandas.
- It is very simple to perform pivoting and reshaping of data sets in Pandas.
- It also provides indexing, label-based slicing, and sub-setting of large data sets.
- We can easily insert and delete columns from a data structure.
- Data aggregation and transformations can be done using group by.
- High-performance merging and joining of data can be done using Pandas.
- It also provides time-series functionality.

Pandas 20 popular and the most important functions:

- **read_csv():** This function reads a CSV file and creates a pandas data frame. It has several options to handle various data formats and separators.
- **head():** This function returns the first n rows of a data frame. By default, it returns the first five rows.
- **tail():** This function returns the last n rows of a data frame. By default, it returns the last five rows.
- **info():** This function provides a summary of a data frame, including the number of rows, columns, data types, and memory usage.
- **describe():** This function provides a statistical summary of a data frame, including the mean, standard deviation, minimum, maximum, and quartile values of each numerical column.

- **dropna():** This function removes rows with missing values from a data frame. It has several options to handle various missing data formats.
- **fillna():** This function fills missing values in a data frame with a specified value or method.
- **groupby():** This function groups a data frame by one or more columns and applies a function to each group.
- **pivot_table():** This function creates a pivot table from a data frame, which summarizes the data by one or more columns.
- **merge():** This function combines two or more data frames based on a common column or index.
- **apply():** This function applies a function to each element of a data frame or series.
- **map():** This function applies a function to each element of a series and returns a new series.
- **value_counts():** This function returns the count of unique values in a series.
- **unique():** This function returns an array of unique values in a series.
- **loc[]:** This function is used to access a group of rows and columns by label(s) or a boolean array.
- **iloc[]:** This function is used to access a group of rows and columns by integer location(s).
- **set_index():** This function sets a column or a list of columns as the index of a data frame.
- **reset_index():** This function resets the index of a data frame to the default integer index.
- **sort_values():** This function sorts a data frame by one or more columns.
- **to_csv():** This function writes a data frame to a CSV file. It has several options to handle various data formats and separators.

In []:



In [1]:



```
!pip install pandas
```

```
Requirement already satisfied: pandas in c:\users\admin\anaconda3\lib\site-packages (1.5.3)
Requirement already satisfied: python-dateutil>=2.8.1 in c:\users\admin\anaconda3\lib\site-packages (from
m pandas) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in c:\users\admin\anaconda3\lib\site-packages (from pandas)
(2022.7)
Requirement already satisfied: numpy>=1.21.0 in c:\users\admin\anaconda3\lib\site-packages (from pandas)
(1.24.3)
Requirement already satisfied: six>=1.5 in c:\users\admin\anaconda3\lib\site-packages (from python-dateu
til>=2.8.1->pandas) (1.16.0)
```

In [1]:



```
import pandas as pd
```

```
In [3]: ▶ lst = [2,5,2,4,6,9]
        lst
```

```
Out[3]: [2, 5, 2, 4, 6, 9]
```

```
In [ ]: ▶
```

```
In [4]: ▶ ser = pd.Series(lst)
        ser
```

```
Out[4]: 0    2
        1    5
        2    2
        3    4
        4    6
        5    9
        dtype: int64
```

```
In [5]: ▶ type(ser)
```

```
Out[5]: pandas.core.series.Series
```

```
In [7]: ▶ ser = pd.Series([4,2,5,6,7,1], index=['a', 'b', 'c', 'd', 'e', 'f'])
        ser
```

```
Out[7]: a    4
        b    2
        c    5
        d    6
        e    7
        f    1
        dtype: int64
```

```
In [9]: a = ['sourav', 'deepak', 'rupesh']  
  
pd.Series([3,4,5], index=a)
```

```
Out[9]: sourav    3  
        deepak    4  
        rupesh    5  
        dtype: int64
```

```
In [12]: ser = pd.Series([4,2,5,6,7,1], index=['a', 'b', 'c', 'd', 'e', 'f'], dtype='float', name='My_Data')  
ser
```

```
Out[12]: a    4.0  
        b    2.0  
        c    5.0  
        d    6.0  
        e    7.0  
        f    1.0  
        Name: My_Data, dtype: float64
```

```
In [ ]: 
```

```
In [13]: ser = pd.Series(15, index=['a', 'b', 'c', 'd', 'e', 'f'], name='Sourav')  
ser
```

```
Out[13]: a    15  
        b    15  
        c    15  
        d    15  
        e    15  
        f    15  
        Name: Sourav, dtype: int64
```

```
In [ ]: 
```

Dataframe

Using List

```
In [14]: ▶ lst_of_lst = [[1,2], [2,3], [3,4]]
```

```
In [16]: ▶ lst_of_lst[1]
```

```
Out[16]: [2, 3]
```

```
In [17]: ▶ lst_of_lst[1][0]
```

```
Out[17]: 2
```

```
In [18]: ▶ lst_of_lst = [['apple', 6], ['banana', 12], ['mango', 5]]  
lst_of_lst
```

```
Out[18]: [['apple', 6], ['banana', 12], ['mango', 5]]
```

```
In [19]: ▶ df = pd.DataFrame(lst_of_lst)  
df
```

```
Out[19]:
```

	0	1
0	apple	6
1	banana	12
2	mango	5

```
In [20]: ► df = pd.DataFrame(lst_of_lst, columns=['Fruits', 'Quantity'])
df
```

```
Out[20]:
```

	Fruits	Quantity
0	apple	6
1	banana	12
2	mango	5

```
In [24]: ► lst_of_lst = [['apple', 6, 'L'], ['banana', 12, 'D'], ['mango', 5, 'L'], ['orange', 10, 'D']]
df = pd.DataFrame(lst_of_lst, columns=['Fruits', 'Quantity', 'Like/Dislike'])
df
```

```
Out[24]:
```

	Fruits	Quantity	Like/Dislike
0	apple	6	L
1	banana	12	D
2	mango	5	L
3	orange	10	D

```
In [25]: ► type(df)
```

```
Out[25]: pandas.core.frame.DataFrame
```

```
In [ ]: ►
```

Using Dictionary

```
In [26]: ► d1 = {'A': 1, 'B': 2}
d1
```

```
Out[26]: {'A': 1, 'B': 2}
```

```
In [27]: d1 = {'A': [1, 3, 5], 'B': [2, 4, 6]}
d1
```

```
Out[27]: {'A': [1, 3, 5], 'B': [2, 4, 6]}
```

```
In [ ]:
```

```
In [30]: d1 = {
    'Fruits': ['apple', 'banana', 'mango', 'orange'],
    'Quantity': [6, 12, 5, 10],
    'Like/Dislike': ['L', 'D', 'L', 'D']
}

df = pd.DataFrame(d1)
df
```

```
Out[30]:
```

	Fruits	Quantity	Like/Dislike
0	apple	6	L
1	banana	12	D
2	mango	5	L
3	orange	10	D

```
In [ ]:
```

```
In [33]: df['Quantity']
```

```
Out[33]: 0    6
1    12
2     5
3    10
Name: Quantity, dtype: int64
```

```
In [34]: df.Quantity
```

```
Out[34]: 0      6
         1     12
         2      5
         3     10
         Name: Quantity, dtype: int64
```

```
In [ ]:
```

```
In [36]: df['Price'] = ['5$', '3$', '6$', '4$']
```

```
In [37]: df
```

```
Out[37]:
```

	Fruits	Quantity	Like/Dislike	Price
0	apple	6	L	5\$
1	banana	12	D	3\$
2	mango	5	L	6\$
3	orange	10	D	4\$

```
In [38]: df.shape
```

```
Out[38]: (4, 4)
```

```
In [39]: df.columns
```

```
Out[39]: Index(['Fruits', 'Quantity', 'Like/Dislike', 'Price'], dtype='object')
```

```
In [ ]:
```

Arithmetic Operation


```
In [41]: ► d1 = pd.DataFrame({'A': [1, 3, 5], 'B': [2, 4, 6]})  
d1
```

```
Out[41]:
```

	A	B
0	1	2
1	3	4
2	5	6

```
In [42]: ► d1['A']
```

```
Out[42]: 0    1  
1    3  
2    5  
Name: A, dtype: int64
```

```
In [43]: ► d1.B
```

```
Out[43]: 0    2  
1    4  
2    6  
Name: B, dtype: int64
```

```
In [44]: ► d1
```

```
Out[44]:
```

	A	B
0	1	2
1	3	4
2	5	6

```
In [49]: d1['A'] * d1['B']
```

```
Out[49]: 0      2  
         1     12  
         2     30  
         dtype: int64
```

```
In [46]: d1['C'] = d1['A'] * d1['B']
```

```
In [47]: d1
```

```
Out[47]:
```

	A	B	C
0	1	2	2
1	3	4	12
2	5	6	30

```
In [ ]:
```

Insert and Delete

```
In [50]: d1
```

```
Out[50]:
```

	A	B	C
0	1	2	2
1	3	4	12
2	5	6	30

```
In [ ]: # insert(index_value, col_name, data)
```

```
In [51]: d1.insert(1, 'Alphabet', ['a', 'b', 'c'])
```

```
In [52]: d1
```

```
Out[52]:
```

	A	Alphabet	B	C
0	1	a	2	2
1	3	b	4	12
2	5	c	6	30

```
In [53]: d1.insert(3, 'Nested_lst', [[1,2], [3,4], [4,6]])
```

```
In [54]: d1
```

```
Out[54]:
```

	A	Alphabet	B	Nested_lst	C
0	1	a	2	[1, 2]	2
1	3	b	4	[3, 4]	12
2	5	c	6	[4, 6]	30

```
In [56]: d1.insert(5, 'Dictionary', [{'name': ['Sourav', 'Deepak']}, {'name': ['Rupesh', 'Raj']}, {'name': ['Himanshu', 'Shubham']}])
```

```
In [57]: d1
```

```
Out[57]:
```

	A	Alphabet	B	Nested_lst	C	Dictionary
0	1	a	2	[1, 2]	2	{'name': ['Sourav', 'Deepak']}
1	3	b	4	[3, 4]	12	{'name': ['Rupesh', 'Raj']}
2	5	c	6	[4, 6]	30	{'name': ['Himanshu', 'Shubham']}

In []: ▶

In []: ▶

Delete

In [58]: ▶

d1

Out[58]:

	A	Alphabet	B	Nested_Ist	C	Dictionary
0	1	a	2	[1, 2]	2	{'name': ['Sourav', 'Deepak']}
1	3	b	4	[3, 4]	12	{'name': ['Rupesh', 'Raj']}
2	5	c	6	[4, 6]	30	{'name': ['Himanshu', 'Shubham']}

In [59]: ▶

del d1['Alphabet']

In [60]: ▶

d1

Out[60]:

	A	B	Nested_Ist	C	Dictionary
0	1	2	[1, 2]	2	{'name': ['Sourav', 'Deepak']}
1	3	4	[3, 4]	12	{'name': ['Rupesh', 'Raj']}
2	5	6	[4, 6]	30	{'name': ['Himanshu', 'Shubham']}

In [64]: ▶

d1.pop('C')

Out[64]:

```
0    2
1   12
2   30
Name: C, dtype: int64
```

In [65]: `d1`

Out[65]:

	A	B	Nested_Ist	Dictionary
0	1	2	[1, 2]	{'name': ['Sourav', 'Deepak']}
1	3	4	[3, 4]	{'name': ['Rupesh', 'Raj']}
2	5	6	[4, 6]	{'name': ['Himanshu', 'Shubham']}

In [66]: `d2 = d1.copy()`

In [67]: `d1`

Out[67]:

	A	B	Nested_Ist	Dictionary
0	1	2	[1, 2]	{'name': ['Sourav', 'Deepak']}
1	3	4	[3, 4]	{'name': ['Rupesh', 'Raj']}
2	5	6	[4, 6]	{'name': ['Himanshu', 'Shubham']}

In [68]: `del d1`

In [69]: `d1`

NameError

Traceback (most recent call last)

Cell In[69], line 1

----> 1 d1

NameError: name 'd1' is not defined

In [70]:

d2

Out[70]:

	A	B	Nested_lst	Dictionary
0	1	2	[1, 2]	{'name': ['Sourav', 'Deepak']}
1	3	4	[3, 4]	{'name': ['Rupesh', 'Raj']}
2	5	6	[4, 6]	{'name': ['Himanshu', 'Shubham']}

In []:

In []:

Reading a CSV file in pandas dataframe

In [77]: `d1 = pd.read_csv('C://Users/Admin/1 - Training Basket Batch_5/petrol_consumption.csv', nrows=11)`
d1

Out[77]:

	Petrol_tax	Average_income	Paved_Highways	Population_Driver_licence(%)	Petrol_Consumption
0	9.0	3571.0	1976.0	0.525	541.0
1	9.0	4092.0	1250.0	0.572	524.0
2	9.0	NaN	NaN	0.580	561.0
3	7.5	4870.0	2351.0	0.529	NaN
4	8.0	4399.0	431.0	0.544	410.0
5	10.0	5342.0	1333.0	0.571	457.0
6	8.0	5319.0	11868.0	0.451	344.0
7	8.0	5126.0	2138.0	0.553	467.0
8	8.0	4447.0	8577.0	0.529	464.0
9	7.0	4512.0	8507.0	0.552	498.0
10	8.0	4391.0	5939.0	0.530	580.0

```
In [78]: data = pd.read_csv('petrol_consumption.csv', nrows=11)
data
```

```
Out[78]:
```

	Petrol_tax	Average_income	Paved_Highways	Population_Driver_licence(%)	Petrol_Consumption
0	9.0	3571.0	1976.0	0.525	541.0
1	9.0	4092.0	1250.0	0.572	524.0
2	9.0	NaN	NaN	0.580	561.0
3	7.5	4870.0	2351.0	0.529	NaN
4	8.0	4399.0	431.0	0.544	410.0
5	10.0	5342.0	1333.0	0.571	457.0
6	8.0	5319.0	11868.0	0.451	344.0
7	8.0	5126.0	2138.0	0.553	467.0
8	8.0	4447.0	8577.0	0.529	464.0
9	7.0	4512.0	8507.0	0.552	498.0
10	8.0	4391.0	5939.0	0.530	580.0

```
In [80]: data = pd.read_csv('petrol_consumption.csv', nrows=6, usecols=['Petrol_tax', 'Petrol_Consumption'])
data
```

```
Out[80]:
```

	Petrol_tax	Petrol_Consumption
0	9.0	541.0
1	9.0	524.0
2	9.0	561.0
3	7.5	NaN
4	8.0	410.0
5	10.0	457.0

In []: ▶

```
In [2]: ▶ data = pd.read_csv('petrol_consumption.csv')  
data
```

Out[2]:

	Petrol_tax	Average_income	Paved_Highways	Population_Driver_licence(%)	Petrol_Consumption
0	9.00	3571.0	1976.0	0.525	541.0
1	9.00	4092.0	1250.0	0.572	524.0
2	9.00	NaN	NaN	0.580	561.0
3	7.50	4870.0	2351.0	0.529	NaN
4	8.00	4399.0	431.0	0.544	410.0
5	10.00	5342.0	1333.0	0.571	457.0
6	8.00	5319.0	11868.0	0.451	344.0
7	8.00	5126.0	2138.0	0.553	467.0
8	8.00	4447.0	8577.0	0.529	464.0
9	7.00	4512.0	8507.0	0.552	498.0
10	8.00	4391.0	5939.0	0.530	580.0

In []: ▶

```
In [ ]: ▶ # Data Analysis
```

```
In [3]: ▶ data.shape
```

Out[3]: (48, 5)


```
In [4]: data.head()
```

Out[4]:

	Petrol_tax	Average_income	Paved_Highways	Population_Driver_licence(%)	Petrol_Consumption
0	9.0	3571.0	1976.0	0.525	541.0
1	9.0	4092.0	1250.0	0.572	524.0
2	9.0	NaN	NaN	0.580	561.0
3	7.5	4870.0	2351.0	0.529	NaN
4	8.0	4399.0	431.0	0.544	410.0

```
In [5]: data.head(3)
```

Out[5]:

	Petrol_tax	Average_income	Paved_Highways	Population_Driver_licence(%)	Petrol_Consumption
0	9.0	3571.0	1976.0	0.525	541.0
1	9.0	4092.0	1250.0	0.572	524.0
2	9.0	NaN	NaN	0.580	561.0

```
In [ ]:
```

```
In [6]: data.tail()
```

Out[6]:

	Petrol_tax	Average_income	Paved_Highways	Population_Driver_licence(%)	Petrol_Consumption
43	7.0	3745.0	2611.0	0.508	591.0
44	6.0	5215.0	2302.0	0.672	782.0
45	9.0	4476.0	3942.0	0.571	510.0
46	7.0	4296.0	4083.0	0.623	610.0
47	7.0	5002.0	9794.0	0.593	524.0

In [7]: `data.tail(8)`

Out[7]:

	Petrol_tax	Average_income	Paved_Highways	Population_Driver_licence(%)	Petrol_Consumption
40	7.0	4449.0	4639.0	0.626	587.0
41	7.0	NaN	3985.0	0.563	699.0
42	7.0	4300.0	3635.0	0.603	632.0
43	7.0	3745.0	2611.0	0.508	591.0
44	6.0	5215.0	2302.0	0.672	782.0
45	9.0	4476.0	3942.0	0.571	510.0
46	7.0	4296.0	4083.0	0.623	610.0
47	7.0	5002.0	9794.0	0.593	524.0

In [8]: `data.ndim`

Out[8]: 2

In [9]: `data.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48 entries, 0 to 47
Data columns (total 5 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Petrol_tax                            48 non-null     float64
1   Average_income                        43 non-null     float64
2   Paved_Highways                        43 non-null     float64
3   Population_Driver_licence(%)         47 non-null     float64
4   Petrol_Consumption                    46 non-null     float64
dtypes: float64(5)
memory usage: 2.0 KB
```

In [10]: `data.describe()`

Out[10]:

	Petrol_tax	Average_income	Paved_Highways	Population_Driver_licence(%)	Petrol_Consumption
count	48.000000	43.000000	43.000000	47.000000	46.000000
mean	7.668333	4261.627907	5126.232558	0.569532	582.608696
std	0.950770	579.517781	2741.972637	0.055788	110.465375
min	5.000000	3063.000000	431.000000	0.451000	344.000000
25%	7.000000	3773.500000	2946.500000	0.529500	524.000000
50%	7.500000	4300.000000	4725.000000	0.563000	572.500000
75%	8.125000	4543.000000	6749.500000	0.589500	634.250000
max	10.000000	5342.000000	11868.000000	0.724000	968.000000

In [13]: `# data.describe(include='object')`

In [15]: `data.describe().transpose()`

Out[15]:

	count	mean	std	min	25%	50%	75%	max
Petrol_tax	48.0	7.668333	0.950770	5.000	7.0000	7.500	8.1250	10.000
Average_income	43.0	4261.627907	579.517781	3063.000	3773.5000	4300.000	4543.0000	5342.000
Paved_Highways	43.0	5126.232558	2741.972637	431.000	2946.5000	4725.000	6749.5000	11868.000
Population_Driver_licence(%)	47.0	0.569532	0.055788	0.451	0.5295	0.563	0.5895	0.724
Petrol_Consumption	46.0	582.608696	110.465375	344.000	524.0000	572.500	634.2500	968.000

```
In [16]: data.describe().T
```

Out[16]:

	count	mean	std	min	25%	50%	75%	max
Petrol_tax	48.0	7.668333	0.950770	5.000	7.0000	7.500	8.1250	10.000
Average_income	43.0	4261.627907	579.517781	3063.000	3773.5000	4300.000	4543.0000	5342.000
Paved_Highways	43.0	5126.232558	2741.972637	431.000	2946.5000	4725.000	6749.5000	11868.000
Population_Driver_licence(%)	47.0	0.569532	0.055788	0.451	0.5295	0.563	0.5895	0.724
Petrol_Consumption	46.0	582.608696	110.465375	344.000	524.0000	572.500	634.2500	968.000

```
In [ ]:
```

Indexing and Slicing

```
In [17]: data
```

35	6.58	3802.0	7834.0	0.629	644.0
36	5.00	4045.0	NaN	0.566	640.0
37	7.00	3897.0	6385.0	0.586	704.0
38	8.50	3635.0	3274.0	0.663	648.0
39	7.00	4345.0	3905.0	0.672	968.0
40	7.00	4449.0	4639.0	0.626	587.0
41	7.00	NaN	3985.0	0.563	699.0
42	7.00	4300.0	3635.0	0.603	632.0
43	7.00	3745.0	2611.0	0.508	591.0
44	6.00	5215.0	2302.0	0.672	782.0
45	9.00	4476.0	3942.0	0.571	510.0
46	7.00	4296.0	4083.0	0.623	610.0
47	7.00	5002.0	9794.0	0.593	524.0

```
In [18]: data[0:3]
```

Out[18]:

	Petrol_tax	Average_income	Paved_Highways	Population_Driver_licence(%)	Petrol_Consumption
0	9.0	3571.0	1976.0	0.525	541.0
1	9.0	4092.0	1250.0	0.572	524.0
2	9.0	NaN	NaN	0.580	561.0

```
In [19]: data[5:11]
```

Out[19]:

	Petrol_tax	Average_income	Paved_Highways	Population_Driver_licence(%)	Petrol_Consumption
5	10.0	5342.0	1333.0	0.571	457.0
6	8.0	5319.0	11868.0	0.451	344.0
7	8.0	5126.0	2138.0	0.553	467.0
8	8.0	4447.0	8577.0	0.529	464.0
9	7.0	4512.0	8507.0	0.552	498.0
10	8.0	4391.0	5939.0	0.530	580.0

```
In [23]: data.iloc[5:11, 0:2]
```

Out[23]:

	Petrol_tax	Average_income
5	10.0	5342.0
6	8.0	5319.0
7	8.0	5126.0
8	8.0	4447.0
9	7.0	4512.0
10	8.0	4391.0

```
In [24]: data.iloc[11:16, 2:]
```

Out[24]:

	Paved_Highways	Population_Driver_licence(%)	Petrol_Consumption
11	NaN	0.525	NaN
12	6930.0	0.574	525.0
13	6580.0	0.545	508.0
14	8159.0	NaN	566.0
15	10340.0	0.586	635.0

```
In [25]: data
```

17	7.00	3718.0	4725.0	0.540	714.0
18	7.00	NaN	5915.0	0.724	865.0
19	8.50	4341.0	6010.0	0.677	640.0
20	7.00	NaN	7834.0	0.663	649.0
21	8.00	4983.0	602.0	0.602	540.0
22	9.00	4897.0	2449.0	0.511	464.0
23	9.00	4258.0	4686.0	0.517	547.0
24	8.50	4574.0	2619.0	0.551	460.0
25	9.00	3721.0	4746.0	0.544	566.0
26	8.00	3448.0	5399.0	0.548	577.0
27	7.50	3846.0	9061.0	0.579	631.0
28	8.00	4188.0	5975.0	0.563	574.0
29	9.00	3601.0	NaN	0.493	534.0

```
In [27]: data.iloc[3:4, 2:3]
```

Out[27]:

	Paved_Highways
3	2351.0

```
In [28]: data.iloc[3:4, 2:3]
```

Out[28]:

	Paved_Highways
3	2351.0

```
In [29]: data.loc[1:5, ['Paved_Highways', 'Petrol_Consumption']]
```

Out[29]:

	Paved_Highways	Petrol_Consumption
1	1250.0	524.0
2	NaN	561.0
3	2351.0	NaN
4	431.0	410.0
5	1333.0	457.0

```
In [ ]:
```

Handling Missing Data

```
In [30]: data.isnull()
```

```
Out[30]:
```

	Petrol_tax	Average_income	Paved_Highways	Population_Driver_licence(%)	Petrol_Consumption
0	False	False	False	False	False
1	False	False	False	False	False
2	False	True	True	False	False
3	False	False	False	False	True
4	False	False	False	False	False
5	False	False	False	False	False
6	False	False	False	False	False
7	False	False	False	False	False
8	False	False	False	False	False
9	False	False	False	False	False
10	False	False	False	False	False

```
In [31]: data.isnull().any()
```

```
Out[31]: Petrol_tax          False
Average_income          True
Paved_Highways          True
Population_Driver_licence(%)  True
Petrol_Consumption      True
dtype: bool
```

```
In [32]: data.isnull().sum()
```

```
Out[32]: Petrol_tax          0
Average_income          5
Paved_Highways          5
Population_Driver_licence(%)  1
Petrol_Consumption      2
dtype: int64
```



```
In [36]: data.isna().sum()
```

```
Out[36]: Petrol_tax          0
Average_income          5
Paved_Highways          5
Population_Driver_licence(%)  1
Petrol_Consumption      2
dtype: int64
```

```
In [ ]:
```

```
In [ ]: # fillna() and dropna()
```

```
In [37]: data
```

35	6.58	3802.0	7834.0	0.629	644.0
36	5.00	4045.0	NaN	0.566	640.0
37	7.00	3897.0	6385.0	0.586	704.0
38	8.50	3635.0	3274.0	0.663	648.0
39	7.00	4345.0	3905.0	0.672	968.0
40	7.00	4449.0	4639.0	0.626	587.0
41	7.00	NaN	3985.0	0.563	699.0
42	7.00	4300.0	3635.0	0.603	632.0
43	7.00	3745.0	2611.0	0.508	591.0
44	6.00	5215.0	2302.0	0.672	782.0
45	9.00	4476.0	3942.0	0.571	510.0
46	7.00	4296.0	4083.0	0.623	610.0
47	7.00	5002.0	9794.0	0.593	524.0

```
In [38]: data.fillna(55)
```

Out[38]:

	Petrol_tax	Average_income	Paved_Highways	Population_Driver_licence(%)	Petrol_Consumption
0	9.00	3571.0	1976.0	0.525	541.0
1	9.00	4092.0	1250.0	0.572	524.0
2	9.00	55.0	55.0	0.580	561.0
3	7.50	4870.0	2351.0	0.529	55.0
4	8.00	4399.0	431.0	0.544	410.0
5	10.00	5342.0	1333.0	0.571	457.0
6	8.00	5319.0	11868.0	0.451	344.0
7	8.00	5126.0	2138.0	0.553	467.0
8	8.00	4447.0	8577.0	0.529	464.0
9	7.00	4512.0	8507.0	0.552	498.0
10	8.00	4391.0	5939.0	0.530	580.0

```
In [40]: data.fillna('Sourav')
```

Out[40]:

	Petrol_tax	Average_income	Paved_Highways	Population_Driver_licence(%)	Petrol_Consumption
0	9.00	3571.0	1976.0	0.525	541.0
1	9.00	4092.0	1250.0	0.572	524.0
2	9.00	Sourav	Sourav	0.58	561.0
3	7.50	4870.0	2351.0	0.529	Sourav
4	8.00	4399.0	431.0	0.544	410.0
5	10.00	5342.0	1333.0	0.571	457.0
6	8.00	5319.0	11868.0	0.451	344.0
7	8.00	5126.0	2138.0	0.553	467.0
8	8.00	4447.0	8577.0	0.529	464.0
9	7.00	4512.0	8507.0	0.552	498.0
10	8.00	4391.0	5939.0	0.53	580.0

```
In [42]: data['Petrol_Consumption'].fillna('5555555')
```

```
Out[42]: 0      541.0  
1      524.0  
2      561.0  
3      5555555  
4      410.0  
5      457.0  
6      344.0  
7      467.0  
8      464.0  
9      498.0  
10     580.0  
11     5555555  
12     525.0  
13     508.0  
14     566.0  
15     635.0  
16     603.0  
17     714.0  
18     865.0  
19     640.0
```

```
In [44]: data['Average_income'].mean()
```

```
Out[44]: 4261.627906976744
```

```
In [46]: data['Average_income'].fillna(data['Average_income'].mean())
```

```
Out[46]: 0    3571.000000  
1    4092.000000  
2    4261.627907  
3    4870.000000  
4    4399.000000  
5    5342.000000  
6    5319.000000  
7    5126.000000  
8    4447.000000  
9    4512.000000  
10   4391.000000  
11   5126.000000  
12   4817.000000  
13   4207.000000  
14   4332.000000  
15   4318.000000  
16   4206.000000  
17   3718.000000  
18   4261.627907  
19   4241.000000
```

```
In [48]: data['Average_income'].fillna(data['Average_income'].mean(), inplace=True)
```

In [49]: `data`

Out[49]:

	Petrol_tax	Average_income	Paved_Highways	Population_Driver_licence(%)	Petrol_Consumption
0	9.00	3571.000000	1976.0	0.525	541.0
1	9.00	4092.000000	1250.0	0.572	524.0
2	9.00	4261.627907	NaN	0.580	561.0
3	7.50	4870.000000	2351.0	0.529	NaN
4	8.00	4399.000000	431.0	0.544	410.0
5	10.00	5342.000000	1333.0	0.571	457.0
6	8.00	5319.000000	11868.0	0.451	344.0
7	8.00	5126.000000	2138.0	0.553	467.0
8	8.00	4447.000000	8577.0	0.529	464.0
9	7.00	4512.000000	8507.0	0.552	498.0
10	8.00	4391.000000	5939.0	0.530	580.0

In [51]: `data.isna().sum()`

Out[51]:

Petrol_tax	0
Average_income	0
Paved_Highways	5
Population_Driver_licence(%)	1
Petrol_Consumption	2
dtype:	int64

In []: `# Dropna()`

```
In [56]: data = data.dropna()  
data
```

Out[56]:

	Petrol_tax	Average_income	Paved_Highways	Population_Driver_licence(%)	Petrol_Consumption
0	9.00	3571.000000	1976.0	0.525	541.0
1	9.00	4092.000000	1250.0	0.572	524.0
4	8.00	4399.000000	431.0	0.544	410.0
5	10.00	5342.000000	1333.0	0.571	457.0
6	8.00	5319.000000	11868.0	0.451	344.0
7	8.00	5126.000000	2138.0	0.553	467.0
8	8.00	4447.000000	8577.0	0.529	464.0
9	7.00	4512.000000	8507.0	0.552	498.0
10	8.00	4391.000000	5939.0	0.530	580.0
12	7.00	4817.000000	6930.0	0.574	525.0
13	7.00	4207.000000	6580.0	0.545	508.0
15	7.00	4318.000000	10340.0	0.586	635.0

```
In [58]: data.shape
```

Out[58]: (41, 5)

```
In [59]: data.isna().sum()
```

Out[59]:

Petrol_tax	0
Average_income	0
Paved_Highways	0
Population_Driver_licence(%)	0
Petrol_Consumption	0
dtype:	int64

In []: ▶

In []: ▶

Duplicate

In [60]: ▶

`data.duplicated()`

Out[60]:

```
0    False
1    False
4    False
5    False
6    False
7    False
8    False
9    False
10   False
12   False
13   False
15   False
17   False
18   False
19   False
20   False
21   False
22   False
23   False
24   False
```

In [61]: ▶

`data.duplicated().sum()`

Out[61]: 0

In [62]: ▶

`data[data.duplicated()]`

Out[62]:

<u>Petrol_tax</u>	<u>Average_income</u>	<u>Paved_Highways</u>	<u>Population_Driver_licence(%)</u>	<u>Petrol_Consumption</u>
-------------------	-----------------------	-----------------------	-------------------------------------	---------------------------

In []: ▶

Join - Merge - Concat

```
In [64]: ▶ d1 = pd.DataFrame({'A': [1,2,3,4,5], 'B': [5,6,7,8,9]})  
d1
```

Out[64]:

	A	B
0	1	5
1	2	6
2	3	7
3	4	8
4	5	9

```
In [69]: ▶ # d2 = pd.DataFrame({'C': [11,12,13,14,15], 'D': [15,16,17,18,19]})  
# d2
```

```
In [70]: ▶ d2 = pd.DataFrame({'C': [11,12,13], 'D': [15,16,17]})  
d2
```

Out[70]:

	C	D
0	11	15
1	12	16
2	13	17

```
In [71]: ▶ dff = d1.join(d2)
dff
```

Out[71]:

	A	B	C	D
0	1	5	11.0	15.0
1	2	6	12.0	16.0
2	3	7	13.0	17.0
3	4	8	NaN	NaN
4	5	9	NaN	NaN

```
In [72]: ▶ dff = d1.join(d2, how='inner')
dff
```

Out[72]:

	A	B	C	D
0	1	5	11	15
1	2	6	12	16
2	3	7	13	17

```
In [73]: ▶ dff = d1.join(d2, how='left')
dff
```

Out[73]:

	A	B	C	D
0	1	5	11.0	15.0
1	2	6	12.0	16.0
2	3	7	13.0	17.0
3	4	8	NaN	NaN
4	5	9	NaN	NaN

```
In [74]: ▶ dff = d1.join(d2, how='right')  
dff
```

Out[74]:

	A	B	C	D
0	1	5	11	15
1	2	6	12	16
2	3	7	13	17

```
In [75]: ▶ dff = d1.join(d2, how='outer')  
dff
```

Out[75]:

	A	B	C	D
0	1	5	11.0	15.0
1	2	6	12.0	16.0
2	3	7	13.0	17.0
3	4	8	NaN	NaN
4	5	9	NaN	NaN

```
In [ ]: ▶
```

```
In [ ]: ▶
```

```
In [ ]: ▶ # Merge
```

```
In [76]: ▶ d1 = pd.DataFrame({'A': [1,2,3,4,5], 'B': [5,6,7,8,9]})  
d2 = pd.DataFrame({'C': [11,12,13,14,15], 'D': [15,16,17,18,19]})  
  
d1
```

Out[76]:

	A	B
0	1	5
1	2	6
2	3	7
3	4	8
4	5	9

```
In [77]: ▶ d2
```

Out[77]:

	C	D
0	11	15
1	12	16
2	13	17
3	14	18
4	15	19

```
In [78]: d1.merge(d2)
```

MergeError

Traceback (most recent call last)

Cell In[78], line 1

----> 1 d1.merge(d2)

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:10093, in DataFrame.merge(self, right, how, on, left_on, right_on, left_index, right_index, sort, suffixes, copy, indicator, validate)

```
10074 @Substitution("")
10075 @Appender(_merge_doc, indents=2)
10076 def merge(
10077     (...)
10089     validate: str | None = None,
10090 ) -> DataFrame:
10091     from pandas.core.reshape.merge import merge
> 10093     return merge(
10094         self,
10095         right,
10096         how=how,
10097         on=on,
10098         left_on=left_on,
10099         right_on=right_on,
10100         left_index=left_index,
10101         right_index=right_index,
10102         sort=sort,
10103         suffixes=suffixes,
10104         copy=copy,
10105         indicator=indicator,
10106         validate=validate,
10107     )
```

```
In [ ]:
```

```
In [81]: d1 = pd.DataFrame({'A': [1,2,3,4,5], 'B': [5,6,7,8,9]})
d2 = pd.DataFrame({'A': [1,2,3,4,5], 'C': [15,16,17,18,19]})
```

```
In [82]: d1.merge(d2, on='A')
```

Out[82]:

	A	B	C
0	1	5	15
1	2	6	16
2	3	7	17
3	4	8	18
4	5	9	19

```
In [83]: d1.merge(d2)
```

Out[83]:

	A	B	C
0	1	5	15
1	2	6	16
2	3	7	17
3	4	8	18
4	5	9	19

```
In [84]: d2.merge(d1)
```

Out[84]:

	A	C	B
0	1	15	5
1	2	16	6
2	3	17	7
3	4	18	8
4	5	19	9

```
In [ ]:
```

```
In [ ]: # Concat
```

```
In [85]: d1 = pd.DataFrame({'A': [1,2,3,4,5], 'B': [5,6,7,8,9]})  
d2 = pd.DataFrame({'C': [11,12,13,14,15], 'D': [15,16,17,18,19]})
```

```
In [87]: ► df1 = pd.concat([d1, d2], axis=0)
df1
```

Out[87]:

	A	B	C	D
0	1.0	5.0	NaN	NaN
1	2.0	6.0	NaN	NaN
2	3.0	7.0	NaN	NaN
3	4.0	8.0	NaN	NaN
4	5.0	9.0	NaN	NaN
0	NaN	NaN	11.0	15.0
1	NaN	NaN	12.0	16.0
2	NaN	NaN	13.0	17.0
3	NaN	NaN	14.0	18.0
4	NaN	NaN	15.0	19.0

```
In [88]: ► df1 = pd.concat([d1, d2], axis=1)
df1
```

Out[88]:

	A	B	C	D
0	1	5	11	15
1	2	6	12	16
2	3	7	13	17
3	4	8	14	18
4	5	9	15	19

```
In [90]: ► df1 = pd.concat([d1, d2], axis=0, ignore_index=True)
df1
```

Out[90]:

	A	B	C	D
0	1.0	5.0	NaN	NaN
1	2.0	6.0	NaN	NaN
2	3.0	7.0	NaN	NaN
3	4.0	8.0	NaN	NaN
4	5.0	9.0	NaN	NaN
5	NaN	NaN	11.0	15.0
6	NaN	NaN	12.0	16.0
7	NaN	NaN	13.0	17.0
8	NaN	NaN	14.0	18.0
9	NaN	NaN	15.0	19.0

```
In [ ]: ►
```

Groupby()


```
In [91]: ▶ import pandas as pd
import numpy as np

# Generate synthetic data
np.random.seed(0)
num_rows = 100
cities = ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Phoenix']
data = {
    'emp_id': np.arange(1, num_rows + 1),
    'emp_name': ['Employee' + str(i) for i in range(1, num_rows + 1)],
    'emp_city': np.random.choice(cities, num_rows),
    'emp_salary': np.random.randint(40000, 120000, num_rows)
}

# Create DataFrame
df = pd.DataFrame(data)
df
```

Out[91]:

	emp_id	emp_name	emp_city	emp_salary
0	1	Employee1	Phoenix	118778
1	2	Employee2	New York	76223
2	3	Employee3	Houston	101570
3	4	Employee4	Houston	46521
4	5	Employee5	Houston	96894
...	...	...	...	...
95	96	Employee96	Los Angeles	78429
96	97	Employee97	New York	116323
97	98	Employee98	Chicago	100809
98	99	Employee99	Chicago	42121
99	100	Employee100	Houston	85353

100 rows × 4 columns

```
In [93]: ▶ df[df['emp_city'] == 'Chicago']
```

Out[93]:

	emp_id	emp_name	emp_city	emp_salary
7	8	Employee8	Chicago	91939
12	13	Employee13	Chicago	96356
24	25	Employee25	Chicago	53640
36	37	Employee37	Chicago	110381
40	41	Employee41	Chicago	82512
42	43	Employee43	Chicago	110557
51	52	Employee52	Chicago	78395
52	53	Employee53	Chicago	100155
60	61	Employee61	Chicago	55912
66	67	Employee67	Chicago	51052
79	80	Employee80	Chicago	70752

```
In [95]: ▶ df[df['emp_city'] == 'Chicago'][['emp_id', 'emp_city']]
```

40	41	Chicago
42	43	Chicago
51	52	Chicago
52	53	Chicago
60	61	Chicago
66	67	Chicago
79	80	Chicago
87	88	Chicago
89	90	Chicago
92	93	Chicago
93	94	Chicago
97	98	Chicago
98	99	Chicago

```
In [ ]: ▶
```

```
In [96]: ▶ df.groupby('emp_city')
```

```
Out[96]: <pandas.core.groupby.generic.DataFrameGroupBy object at 0x00000191D8C9D5D0>
```

```
In [102]: ► for city, emp_data in df.groupby('emp_city'):
            print(city)
            print()
            print(emp_data)
            print('\n')
            print('-----')
```

Chicago

	emp_id	emp_name	emp_city	emp_salary
7	8	Employee8	Chicago	91939
12	13	Employee13	Chicago	96356
24	25	Employee25	Chicago	53640
36	37	Employee37	Chicago	110381
40	41	Employee41	Chicago	82512
42	43	Employee43	Chicago	110557
51	52	Employee52	Chicago	78395
52	53	Employee53	Chicago	100155
60	61	Employee61	Chicago	55912
66	67	Employee67	Chicago	51052
79	80	Employee80	Chicago	70752
87	88	Employee88	Chicago	105180
89	90	Employee90	Chicago	103003
92	93	Employee93	Chicago	109939
93	94	Employee94	Chicago	57582
97	98	Employee98	Chicago	100809
98	99	Employee99	Chicago	42121

Houston

	emp_id	emp_name	emp_city	emp_salary
2	3	Employee3	Houston	101570
3	4	Employee4	Houston	46521
4	5	Employee5	Houston	96894
6	7	Employee7	Houston	54254
20	21	Employee21	Houston	119128
22	23	Employee23	Houston	48286
25	26	Employee26	Houston	108627
28	29	Employee29	Houston	90586
29	30	Employee30	Houston	112130
30	31	Employee31	Houston	61752
38	39	Employee39	Houston	109069
39	40	Employee40	Houston	108473
58	59	Employee59	Houston	78040
59	60	Employee60	Houston	56975
61	62	Employee62	Houston	68086
63	64	Employee64	Houston	119983
68	69	Employee69	Houston	97368

72	73	Employee73	Houston	41740
78	79	Employee79	Houston	42775
83	84	Employee84	Houston	86774
91	92	Employee92	Houston	83671
99	100	Employee100	Houston	85353

Los Angeles

	emp_id	emp_name	emp_city	emp_salary
5	6	Employee6	Los Angeles	114659
13	14	Employee14	Los Angeles	63306
15	16	Employee16	Los Angeles	118696
16	17	Employee17	Los Angeles	118928
18	19	Employee19	Los Angeles	110838
27	28	Employee28	Los Angeles	43912
32	33	Employee33	Los Angeles	47997
33	34	Employee34	Los Angeles	93006
34	35	Employee35	Los Angeles	80800
48	49	Employee49	Los Angeles	75489
50	51	Employee51	Los Angeles	55588
54	55	Employee55	Los Angeles	40469
55	56	Employee56	Los Angeles	83295
56	57	Employee57	Los Angeles	114253
57	58	Employee58	Los Angeles	74488
65	66	Employee66	Los Angeles	104223
81	82	Employee82	Los Angeles	111892
82	83	Employee83	Los Angeles	73930
86	87	Employee87	Los Angeles	55997
95	96	Employee96	Los Angeles	78429

New York

	emp_id	emp_name	emp_city	emp_salary
1	2	Employee2	New York	76223
9	10	Employee10	New York	63310
10	11	Employee11	New York	71785
14	15	Employee15	New York	76950
17	18	Employee18	New York	119904
21	22	Employee22	New York	73538

23	24	Employee24	New York	58728
26	27	Employee27	New York	85663
31	32	Employee32	New York	99715
35	36	Employee36	New York	55620
43	44	Employee44	New York	57340
44	45	Employee45	New York	41913
46	47	Employee47	New York	53429
53	54	Employee54	New York	78214
62	63	Employee63	New York	55115
76	77	Employee77	New York	87805
80	81	Employee81	New York	119464
84	85	Employee85	New York	68651
85	86	Employee86	New York	102885
90	91	Employee91	New York	74009
94	95	Employee95	New York	69844
96	97	Employee97	New York	116323

Phoenix

	emp_id	emp_name	emp_city	emp_salary
0	1	Employee1	Phoenix	118778
8	9	Employee9	Phoenix	77073
11	12	Employee12	Phoenix	104570
19	20	Employee20	Phoenix	73920
37	38	Employee38	Phoenix	94268
41	42	Employee42	Phoenix	59608
45	46	Employee46	Phoenix	92086
47	48	Employee48	Phoenix	53907
49	50	Employee50	Phoenix	50088
64	65	Employee65	Phoenix	92573
67	68	Employee68	Phoenix	55741
69	70	Employee70	Phoenix	83986
70	71	Employee71	Phoenix	45103
71	72	Employee72	Phoenix	75050
73	74	Employee74	Phoenix	95270
74	75	Employee75	Phoenix	52579
75	76	Employee76	Phoenix	111382
77	78	Employee78	Phoenix	60165
88	89	Employee89	Phoenix	114498

In []: ▶

In [103]: ▶

```
df.groupby('emp_city').sum()
```

C:\Users\Admin\AppData\Local\Temp\ipykernel_17288\3387260357.py:1: FutureWarning: The default value of numeric_only in DataFrameGroupBy.sum is deprecated. In a future version, numeric_only will default to False. Either specify numeric_only or select only columns which should be valid for the function.

```
df.groupby('emp_city').sum()
```

Out[103]:

	emp_id	emp_salary
emp_city		
Chicago	1042	1420285
Houston	1000	1818055
Los Angeles	942	1660195
New York	1062	1706428
Phoenix	1004	1510645

In []: ▶


```
In [104]: df.groupby('emp_city').mean()
```

C:\Users\Admin\AppData\Local\Temp\ipykernel_17288\731554166.py:1: FutureWarning: The default value of numeric_only in DataFrameGroupBy.mean is deprecated. In a future version, numeric_only will default to False. Either specify numeric_only or select only columns which should be valid for the function.

```
df.groupby('emp_city').mean()
```

Out[104]:

	emp_id	emp_salary
emp_city		
Chicago	61.294118	83546.176471
Houston	45.454545	82638.863636
Los Angeles	47.100000	83009.750000
New York	48.272727	77564.909091
Phoenix	52.842105	79507.631579

```
In [ ]:
```

```
In [105]: df.groupby('emp_city').min()
```

Out[105]:

	emp_id	emp_name	emp_salary
emp_city			
Chicago	8	Employee13	42121
Houston	3	Employee100	41740
Los Angeles	6	Employee14	40469
New York	2	Employee10	41913
Phoenix	1	Employee1	45103

```
In [106]: df.groupby('emp_city').max()
```

Out[106]:

	emp_id	emp_name	emp_salary
emp_city			
Chicago	99	Employee99	110557
Houston	100	Employee92	119983
Los Angeles	96	Employee96	118928
New York	97	Employee97	119904
Phoenix	89	Employee9	118778

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [107]: # Apply groupby function on emp_city
grouped_df = df.groupby('emp_city').agg({
    'emp_id': 'count',
    'emp_salary': ['mean', 'median', 'min', 'max']
}).reset_index()

# Rename columns for clarity
grouped_df.columns = ['emp_city', 'count', 'avg_salary', 'median_salary', 'min_salary', 'max_salary']

grouped_df
```

Out[107]:

	emp_city	count	avg_salary	median_salary	min_salary	max_salary
0	Chicago	17	83546.176471	91939.0	42121	110557
1	Houston	22	82638.863636	86063.5	41740	119983
2	Los Angeles	20	83009.750000	79614.5	40469	118928
3	New York	22	77564.909091	73773.5	41913	119904
4	Phoenix	19	79507.631579	77073.0	45103	118778

In []:

```
In [108]: df
```

```
Out[108]:
```

	emp_id	emp_name	emp_city	emp_salary
0	1	Employee1	Phoenix	118778
1	2	Employee2	New York	76223
2	3	Employee3	Houston	101570
3	4	Employee4	Houston	46521
4	5	Employee5	Houston	96894
...	...	...	...	...
95	96	Employee96	Los Angeles	78429
96	97	Employee97	New York	116323
97	98	Employee98	Chicago	100809
98	99	Employee99	Chicago	42121
99	100	Employee100	Houston	85353

100 rows × 4 columns

```
In [109]: df['emp_city'].nunique()
```

```
Out[109]: 5
```

```
In [110]: df['emp_city'].unique()
```

```
Out[110]: array(['Phoenix', 'New York', 'Houston', 'Los Angeles', 'Chicago'],  
              dtype=object)
```

```
In [111]: df['emp_city'].value_counts()
```

```
Out[111]: New York      22
Houston      22
Los Angeles  20
Phoenix      19
Chicago      17
Name: emp_city, dtype: int64
```

```
In [ ]:
```

```
In [112]: df
```

```
Out[112]:
```

	emp_id	emp_name	emp_city	emp_salary
0	1	Employee1	Phoenix	118778
1	2	Employee2	New York	76223
2	3	Employee3	Houston	101570
3	4	Employee4	Houston	46521
4	5	Employee5	Houston	96894
...	...	...	...	...
95	96	Employee96	Los Angeles	78429
96	97	Employee97	New York	116323
97	98	Employee98	Chicago	100809
98	99	Employee99	Chicago	42121
99	100	Employee100	Houston	85353

100 rows × 4 columns

```
In [118]: df.to_csv('employee_data.csv', index=False)
```

In []:

In []:

In [119]:

pd.read_csv('employee_data.csv')

Out[119]:

	emp_id	emp_name	emp_city	emp_salary
0	1	Employee1	Phoenix	118778
1	2	Employee2	New York	76223
2	3	Employee3	Houston	101570
3	4	Employee4	Houston	46521
4	5	Employee5	Houston	96894
...	...	...	...	...
95	96	Employee96	Los Angeles	78429
96	97	Employee97	New York	116323
97	98	Employee98	Chicago	100809
98	99	Employee99	Chicago	42121
99	100	Employee100	Houston	85353

100 rows × 4 columns

In []:

In []:

In []:

